

This lecture discusses different types of arguments in Python functions: positional, keyword, and default arguments.

- **Positional Arguments:** These are arguments passed to a function based on their order or position. The function maps the values to the parameters in the order they are provided. For example, in `test(a, b)`, if you call `test(23, 45)`, `a` will be assigned 23 and `b` will be assigned 45 based on their position.
- **Keyword Arguments:** These arguments are passed with explicit parameter names. This allows you to pass arguments in any order, as the function identifies the correct parameter based on the keyword. For example, `test(b=45, a=23)` will correctly assign 23 to `a` and 45 to `b`, regardless of the order.
- **Default Arguments:** These are arguments that have a default value assigned in the function definition. If a value for a default argument is not provided when calling the function, the default value is used. For example, in `test2(a, b, c, d=10)`, `d` has a default value of 10. If you call `test2(2, 3, 5)`, `a` will be 2, `b` will be 3, `c` will be 5, and `d` will default to 10. You can override the default value by explicitly passing a new value, such as `test2(2, 3, 5, 50)`, where `d` will now be 50.

The main takeaways are:

- Python functions can accept arguments in different ways: by position, by keyword, or with default values.
- Positional arguments rely on the order in which they are passed.
- Keyword arguments allow you to specify the parameter name, enabling out-of-order passing.
- Default arguments provide a fallback value if an argument is not explicitly provided during the function call.
-